PL/SQL Training

SESSION 6: Using Explicit Cursors & Triggers

DAVA.X ACADEMY SESSIONS

Svetlana Sura & Iulian Aurel Cozma

10–20 JUNE 2025

SESSION 6

Using Explicit Cursors

- Distinguish between an implicit and an explicit cursor
- Discuss when and why to use an explicit cursor
- Declare and control explicit cursors
- Use simple loop and cursor `FOR` loop to fetch data
- Declare and use cursors with parameters
- Lock rows using the `FOR UPDATE` clause
- Reference the current row with the `WHERE CURRENT` clause

About Cursors

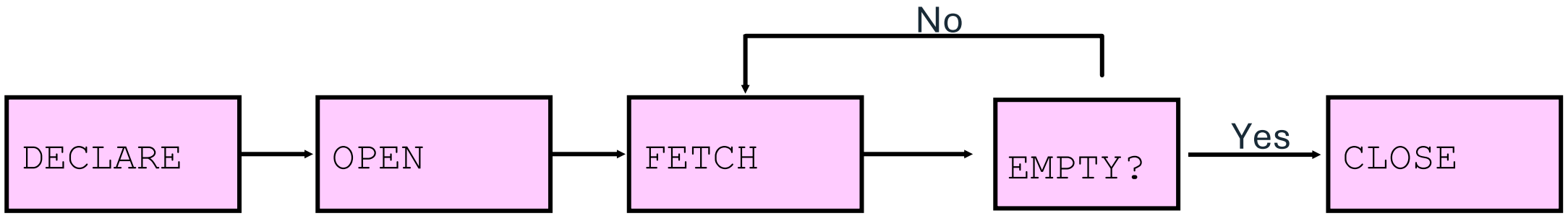
Every SQL statement executed by the Oracle Server has an individual cursor associated with it:

- Implicit cursors: Declared and managed by PL/SQL for all DML and PL/SQL `SELECT` statements
- Explicit cursors: Declared and managed by the programmer

An explicit cursor in PL/SQL is a cursor that you define and control manually for processing multiple rows returned by a query. Unlike implicit cursors (which handle only one row at a time and are managed automatically by Oracle), explicit cursors give you full control over the lifecycle: DECLARE → OPEN → FETCH → CLOSE.

Controlling Explicit Cursors

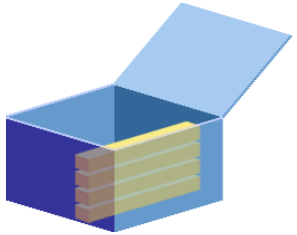
Controlling the flow of explicit cursors in Oracle PL/SQL involves managing the cursor's lifecycle using four key steps: DECLARE, OPEN, FETCH, and CLOSE. Explicit cursors are user-defined cursors used for row-by-row processing of query results when you need fine-grained control over iteration.



- Create a named SQL area
- Identify the active set
- Load the current row into variables
- Test for existing rows
- Return to `FETCH` if rows are found
- Release the active set

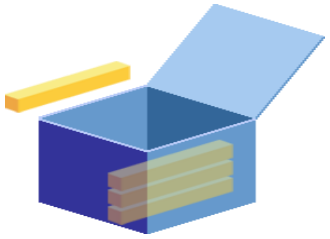
Controlling Explicit Cursors

1 Open the cursor.



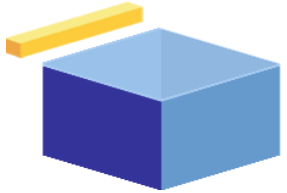
Cursor pointer

2 Fetch a row.



Cursor pointer

3 Close the cursor.



Cursor pointer

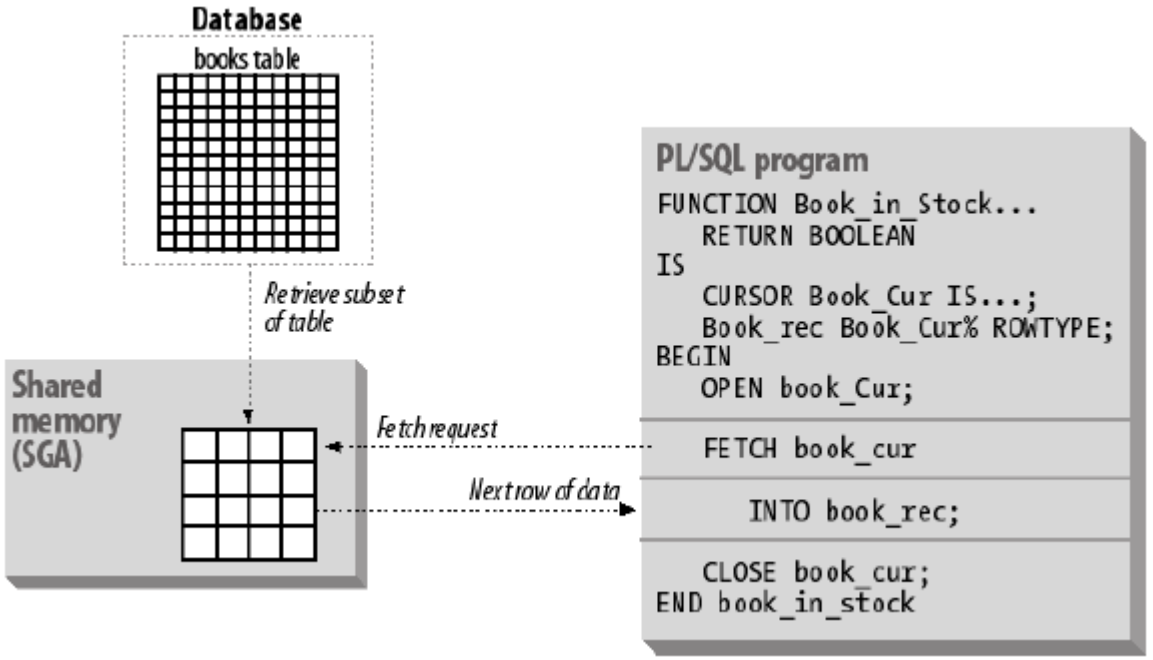
Cursors

Open – The first step allocates the context area for carrying out further processing.

Parse – Next step in processing a SQL statement is to parse it to make sure it is valid and to determine the execution plan.

Bind – When you bind, you associate values from your program (host variables) with placeholders inside your SQL statement.

Execute – In the execute phase, the statement is run within the SQL engine. In addition, the record pointer moves to the



Declaring the Cursor

endava

Syntax:

```
CURSOR cursor_name IS
     select_statement;
```

Examples:

```
DECLARE
  CURSOR emp_cursor IS
  SELECT employee_id, last_name FROM employees
  WHERE department_id =30;
```

```
DECLARE
  CURSOR emp_cursor IS
  SELECT employee_id, last_name FROM employees
  WHERE department_id =30;
```

Opening the Cursor

```
DECLARE
  CURSOR emp_cursor IS
   SELECT employee_id, last_name FROM employees
   WHERE department_id =30;
...
BEGIN
  OPEN emp_cursor;
```

Fetching Data from the Cursor

```
SET SERVEROUTPUT ON
DECLARE
  CURSOR emp_cursor IS
   SELECT employee_id, last_name FROM employees
   WHERE department_id =30;

  empno employees.employee_id%TYPE;
  lname employees.last_name%TYPE;

BEGIN
  OPEN emp_cursor;
  FETCH emp_cursor INTO empno, lname;
  DBMS_OUTPUT.PUT_LINE( empno ||' '||lname);
  ...
END;
/
```

Fetching Data from the Cursor

```
SET SERVEROUTPUT ON
DECLARE
  CURSOR emp_cursor IS
   SELECT employee_id, last_name FROM employees
   WHERE  department_id =30;
  empno employees.employee_id%TYPE;
  lname employees.last_name%TYPE;
BEGIN
  OPEN emp_cursor;
  LOOP
    FETCH emp_cursor INTO empno, lname;
    EXIT WHEN emp_cursor%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE( empno ||' '||lname);
  END LOOP;
  ...
END;
/
```

Closing the Cursor

```
...
  LOOP
    FETCH emp_cursor INTO empno, lname;
    EXIT WHEN emp_cursor%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE( empno ||' '||lname);
  END LOOP;
 CLOSE emp_cursor;
END;
/
```

Cursors and Records

When working with explicit cursors in PL/SQL, it's common to use records to fetch entire rows conveniently.

A record is a composite variable that can hold a row of data — ideal when your cursor query returns multiple columns.

Process the rows of the active set by fetching values into a PL/SQL `RECORD`.

🧩 Why Use Records with Cursors?

- Simplifies variable declarations
- Cleaner syntax for FETCH operations
- Reduces chances of data type mismatches
- Improves readability and maintainability

```
DECLARE
  CURSOR emp_cursor IS
   SELECT employee_id, last_name
   FROM employees
   WHERE  department_id =30;
   emp_record  emp_cursor%ROWTYPE;
BEGIN
  OPEN emp_cursor;
  LOOP
    FETCH emp_cursor INTO emp_record;
  ...
```

Cursor FOR Loops

Syntax:

```
FOR record_name IN cursor_name LOOP
  statement1;
  statement2;
  . . .
END LOOP;
```

The cursor `FOR` loop is a shortcut to process explicit cursors.
Implicit open, fetch, exit, and close occur.
The record is implicitly declared.

Cursor Attributes for Flow Control

endava

🧠 Cursor Attributes for Flow Control

Attribute	Type	Description
cursor_name%ISOPEN	Boolean	Returns TRUE if the cursor is open.
cursor_name%FOUND	Boolean	Returns TRUE if the last fetch returned a row.
cursor_name%NOTFOUND	Boolean	Returns TRUE if the last fetch failed to return a row.
cursor_name%ROWCOUNT	Number	Returns the number of rows fetched so far.

The %ISOPEN Attribute

Fetch rows only when the cursor is open.

Use the %ISOPEN cursor attribute before performing a fetch to test whether the cursor is open.

Example:

```
IF NOT emp_cursor%ISOPEN THEN
    OPEN emp_cursor;
END IF;
LOOP
  FETCH emp_cursor...
```

Example of %ROWCOUNT and %NOTFOUND

endava

```
SET SERVEROUTPUT ON
DECLARE
  empno  employees.employee_id%TYPE;
  ename  employees.last_name%TYPE;
  CURSOR emp_cursor IS SELECT employee_id,
  last_name FROM employees;
BEGIN
  OPEN emp_cursor;
  LOOP
   FETCH emp_cursor INTO empno, ename;
   EXIT WHEN emp_cursor%ROWCOUNT > 10 OR
                        emp_cursor%NOTFOUND;
   DBMS_OUTPUT.PUT_LINE(TO_CHAR(empno)
                           ||' '|| ename);
  END LOOP;
  CLOSE emp_cursor;
END ;
/
```

Cursors with Parameters

Syntax:

```
CURSOR cursor_name
  [(parameter_name datatype, ...)]
IS
  select_statement;
```

Pass parameter values to a cursor when the cursor is opened and the query is executed.
Open an explicit cursor several times with a different active set each time.

```
OPEN  cursor_name(parameter_value,.....) ;
```

Cursors with Parameters

endava

```
SET SERVEROUTPUT ON
DECLARE
  CURSOR   emp_cursor (deptno NUMBER) IS
   SELECT  employee_id, last_name
   FROM    employees
   WHERE   department_id = deptno;
   dept_id NUMBER;
   lname   VARCHAR2(15);
BEGIN
  OPEN emp_cursor (10);
  ...
  CLOSE emp_cursor;
  OPEN emp_cursor (20);

  ...
```

The FOR UPDATE Clause

The FOR UPDATE clause in an explicit cursor locks the selected rows to prevent other sessions from modifying them until your transaction ends (via COMMIT or ROLLBACK). It's crucial in scenarios where you fetch data with the intent to update it afterward.

Syntax:

```
SELECT      ...
FROM        ...
FOR UPDATE [OF column_reference][NOWAIT | WAIT n | SKIP LOCKED];
```

Where:

- FOR UPDATE – Locks all selected rows
- OF *column_reference* – (Optional) Hints to the DB which columns might be updated
- NOWAIT – Fails immediately if a row is locked by another session
- SKIP LOCKED – Skips locked rows without error

Use explicit locking to deny access to other sessions for the duration of a transaction.
Lock the rows *before* the update or delete.

The WHERE CURRENT OF Clause

Syntax:

```
WHERE CURRENT OF cursor ;
```

Use cursors to update or delete the current row.

Include the `FOR UPDATE` clause in the cursor query to lock the rows first.

Use the `WHERE CURRENT OF` clause to reference the current row from an explicit cursor.

```
UPDATE employees
   SET    salary = ...
   WHERE CURRENT OF emp_cursor;
```

🛠️ Use Cases

1. **Payroll systems:** fetch employees and adjust salaries
2. **Banking systems**: fetch accounts and update balances
3. **Inventory control**: lock and update product quantities

⚠️ **Important**

Point	Details
Lock Duration	Locks are held **until COMMIT or ROLLBACK**
Concurrency	Other sessions trying to update locked rows will wait (or fail with NOWAIT)
Avoid Deadlocks	Use consistent ordering in SELECTs when multiple updates are performed

✅ Summary

Feature	Description
FOR UPDATE	Locks rows for update
OF column_list	Optimizes intent of update
NOWAIT	Fails immediately if rows are locked
SKIP LOCKED	Skips locked rows without error
WHERE CURRENT OF	Targets row fetched by cursor

📌 Best Practices

- Always CLOSE your cursors explicitly to avoid memory leaks.
- Use %NOTFOUND to avoid reading past the last row.
- Use %TYPE or %ROWTYPE to avoid datatype mismatches.
- If you don’t need row-by-row control, prefer implicit cursors or bulk collect for performance.
- SYS_REFCURSOR for returning cursors inside PL/SQL logic.
- Use DBMS_SQL.RETURN_RESULT when building generic result-returning APIs or tools(from Oracle 12c).

📚 Optional Enhancements

- Use cursor FOR loops for cleaner syntax (implicitly handles OPEN, FETCH, CLOSE).
- Use parameterized cursors if dynamic filtering is required.

SESSION 6 :

Triggers

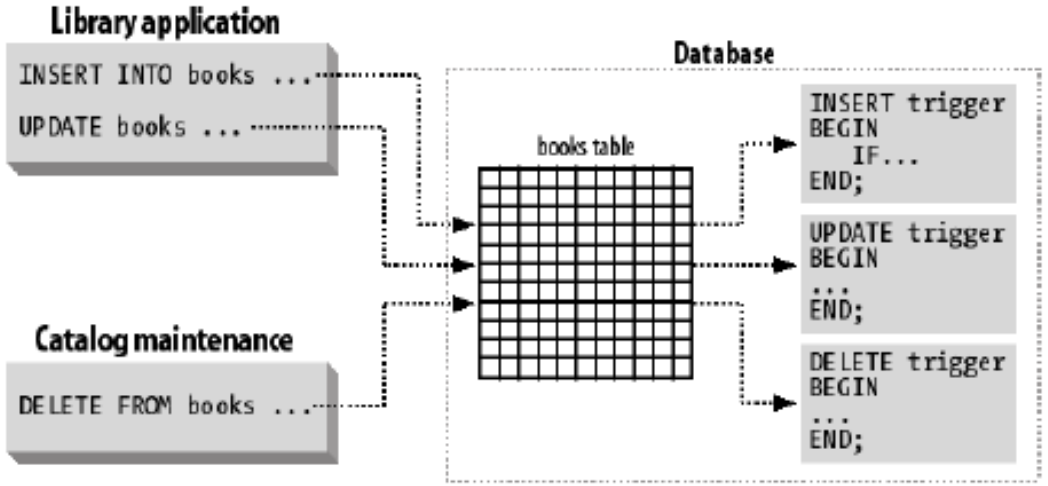
- Types of Triggers
- INSTEAD OF Trigger
- Limitations & Considerations

Triggers

Database triggers are named program units that are executed in response to events that occur in the database. Is a PL/SQL block associated with a table, view, schema or the database.

Trigger types:

- A database manipulat
- A database definition
- A database operation



WN)

Types of Triggers

Scope	Event Triggers On	Description
DML Triggers	Tables, Views	Fired on INSERT, UPDATE, DELETE
Instead-of Triggers	Views	Used to update complex views
System Triggers	Database or Schema	Fired on LOGON, LOGOFF, DDL, STARTUP, etc.
Compound Triggers	Tables	Handle row- and statement-level logic together (Oracle 11g+)

Triggers in Oracle PL/SQL

A trigger in Oracle is a stored PL/SQL block that automatically executes in response to certain events on a table, view, schema, or database — such as INSERT, UPDATE, DELETE, LOGON, or DDL operations.Triggers help enforce business rules, maintain audit trails, or perform automatic validations behind the scenes.

✅ Basic Syntax for DML Trigger

```
CREATE OR REPLACE TRIGGER trigger_name
  BEFORE | AFTER INSERT | UPDATE | DELETE
  ON table_name
  [FOR EACH ROW]
DECLARE
  -- variable declarations
BEGIN
  -- logic to execute
END;
/
```

🔍 Key Concepts

Term	Meaning
BEFORE	Trigger fires before the DML happens
AFTER	Trigger fires after the DML completes
FOR EACH ROW	Row-level trigger (fires for each affected row)
:OLD, :NEW	Pseudorecords holding column values before and after change

Triggers

endava

DML triggers:

```
1   CREATE [OR REPLACE] TRIGGER trigger_name
2   {BEFORE | AFTER | INSTEAD OF}
3   {INSERT | DELETE | UPDATE | UPDATE OF column_list } ON table_name | view_name
4   [FOR EACH ROW]
```

Line	Description
2	Trigger Timing (INSTEAD OF only for views)
3	Triggering event (for INSTEAD OF will use view name)
4	Trigger Type (if line exists is row by row, else statement by statement)
5	Logic to avoid unnecessary execution
6-10	Trigger body

```
10  END [trigger_name];
```

🔌 Disabling / Enabling Triggers

endava

🔁 Disabling and Enabling Triggers, Cursors, and Constraints in Oracle

In Oracle, disabling/enabling is a powerful way to temporarily suspend automatic behavior (like triggers, constraints, or jobs) for maintenance, debugging, or bulk operations.

Below is a guide to what can be enabled/disabled, how to do it, and important precautions.

- **Disable a Trigger**

```
ALTER TRIGGER trg_audit_salary DISABLE;
```

- **Enable a Trigger**

```
ALTER TRIGGER trg_audit_salary ENABLE;
```

- **Enable /Disable All Triggers on a Table**

```
ALTER TABLE employees ENABLE /DISABLE ALL TRIGGERS;
```

Good to know

📌 Best Practices

- Avoid complex logic in triggers to reduce hidden performance overhead.
- Use triggers for auditing, enforcing constraints not supported by SQL, and replicating changes.
- Always document and test thoroughly — cascading triggers can be hard to debug.
- Use compound triggers to improve performance in row-level operations with multiple rows.

⚠️ Limitations & Considerations

- Triggers don’t support COMMIT/ROLLBACK within their body.
- Can lead to mutating table errors (especially in row-level triggers that query the same table).
- Excessive use may reduce transparency and maintainability.

Summary

Trigger Type	Fires On	Scope
BEFORE INSERT	Before row is inserted	Row/Table
AFTER UPDATE	After row is updated	Row/Table
INSTEAD OF	For complex views	Row
COMPOUND	Multiple event phases	Row/Table
SYSTEM	LOGON, LOGOFF, DDL, STARTUP	Database-wide

Summary & Homework

THEORY

PL/SQL 101 Part 12

PL/SQL Tutorial

PRACTICE

- **Different Types of Cursors in PL/SQL**
- **Fetching Multiple Rows from Dynamic SELECT**

HOMEWORK

- **Quiz on Fetching a Single Row**
- **Quiz on Database Triggers**
- **Quiz on The WHEN clause in a trigger**
- **Quiz on Commit and Rollback Inside DML Trigger**